EJ4 – Identificación de paradigmas de desarrollo de IA – Álvaro Fernandez Becerra

Descripción del problema:

El juego del 3 en raya tiene 9 casillas. En teoría hay casi 20 mil tableros distintos, pero muchos no son posibles.

Si analizamos desde la posición inicial, el número de caminos es enorme.

Pero si empezamos cuando ya se han hecho 4 o 5 jugadas, solo quedan 4 o 5 movimientos posibles, y podemos analizar las decisiones del jugador con unas 60 posibilidades en total.

Podemos diseñar reglas heurísticas (como "bloquear al rival" o "buscar la victoria inmediata") sin necesitar un algoritmo de búsqueda completo.

Tarea 1: Escribir las reglas como un jugador experto. (Entregar)

Piensa **como un jugador experto de 3 en raya**, y define al menos 3 reglas **de decisión óptimas**.

Ejemplo:

- **Regla 1:** Si puedo ganar en esta jugada, coloco la ficha en esa casilla.
- **Regla 2:** Si el rival puede ganar en su siguiente jugada, bloqueo esa casilla.
- **Regla 3:** Si el centro está libre, juego en el centro
- **Regla 4:** Si las esquinas están libres, juego en una esquina
- **Regla 5:** Si tengo dos fichas en una línea (fila, columna o diagonal) y la tercera casilla está libre, creo una amenaza doble para forzar la victoria.
- **Regla 6:** Si el rival tiene una amenaza doble (puede ganar por dos vías distintas), bloqueo una de sus amenazas y genero una propia para no quedar en desventaja.

Tarea 2: Modelado simbólico (Entrega)

Representa las jugadas posibles a partir de una posición inicial simplificada usando la plantilla Excel facilitada. Con esta tarea estamos **delimitando el espacio de estados.**

X	O	X
O		

Juega X

1

X	O	X
O	X	

2

X	O	X
O		
X		

3

X	O	X
O		
		X

Juega O

1.1

X	O	X
O	X	O

1.2

X	O	X
O	X	
O		

1.3

X	O	X
O	X	
	O	

1.4

X	O	X
O	X	
		O

2.1

X	O	X
O	O	
X		

2.2

X	O	X
O		O
X		

2.3

X	O	X
O		
X	O	

2.4

X	O	X
O		
X		O

3.1

X	O	X
O	O	
		X

3.2

X	O	X
O		O
		X

3.3

X	O	X
O		
O		X

3.4

X	O	X
O		
	O	X

Juega X

1.1.1

X	O	X
O	X	O
X		

X GANA

1.2.1

X	O	X
O	X	
O		X

X GANA

1.3.1

X	O	X
O	X	
	O	X

X GANA

1.4.1

X	O	X
O	X	
X		O

X GANA

2.1.1

X	O	X
O	O	X

2.2.1

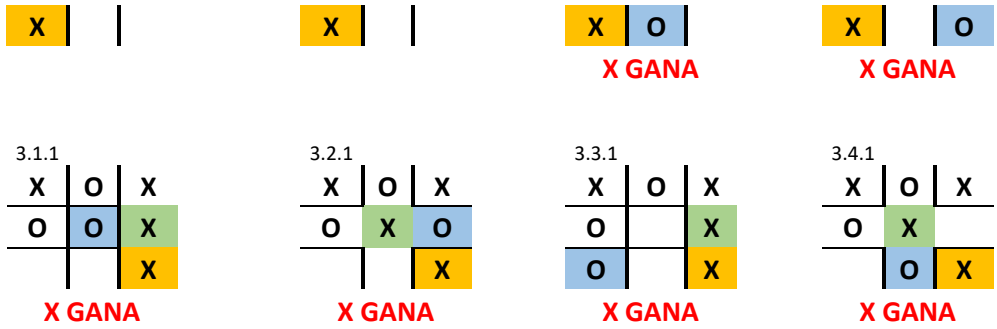
X	O	X
O	X	O

2.3.1

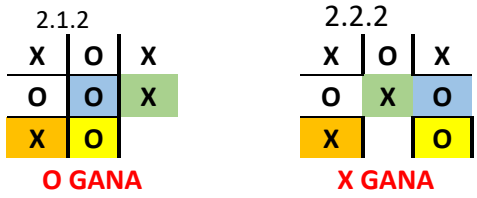
X	O	X
O	X	

2.4.1

X	O	X
O	X	



Juega 0



Tarea 3: Formalización y/o implementación (Entrega)

Programa en Python una función para validar una de las reglas anteriormente definidas por ti.

Parte de una lista de listas en Pyhton

[[X, O, X], [O, ,], [, ,]]

```
def puede_ganar(tablero, jugador):

    # comprobar filas
    for fila in tablero:
        if fila.count(jugador) == 2 and fila.count("") == 1:
            return True

    # comprobar columnas
    for col in range(3):
        columna = [tablero[0][col], tablero[1][col], tablero[2][col]]
        if columna.count(jugador) == 2 and columna.count("") == 1:
            return True

    # diagonal principal
    diag1 = [tablero[0][0], tablero[1][1], tablero[2][2]]
    if diag1.count(jugador) == 2 and diag1.count("") == 1:
        return True
```

```
    # diagonal secundaria
    diag2 = [tablero[0][2], tablero[1][1], tablero[2][0]]
    if diag2.count(jugador) == 2 and diag2.count("") == 1:
        return True

    return False


tablero = [
["X","O","X"],
["O","",""],
["","",""]
]

if puede_ganar(tablero,"O"):
    print("X puede ganar en la siguiente jugada")
else:
    print("No hay victoria inmediata")
```